Tableau Resetmaxxing for Fun and Profit

(a crazy idea that works really well)

Introduction and Novel Idea

Rawson and Reger introduce SATCoP [4], a tableau prover based on LeanCoP [3] that runs a separate SAT solver along fed with grounded atoms from the tableau prover. If the SAT solver reaches an unsatisfiable state, then we can prove that the entire tree is unsatisfiable.

However, other than the underlying SAT solver, the tableau prover in the implementation of SATCoP only has a few improvements over the original LeanCoP.

1. Randomisation of literals at `Start` and `Extension` clauses.
2. Deepening control: only increase maximum height when the SAT model is stale.
3. And a big improvements present in **9.1 A Note from the Future** and in CASC: SAT-guided preference: reorder extensions by the SAT model's opinions of the literals.

I improved the current model of `SATSolve` with the reordering in Item 3 to generate `SATSolveLiteral`. This already provides a big improvement over the baseline.

For the most part, this guidance tries to maximise the chance of closing the tableau rather than trying to create UNSAT on the underlying SAT solver.

While experimenting with choices for SAT-based guidance[2], I found some big improvements in specifically changing the tableau search to achieve this. In particular, **resetting the tableau search early and often to generate different statements for the SAT solver considerably improves the overall model**.

This makes some sense intuitively: instead of trying to close the tableau, we are trying to generate the maximum amount of valid SAT statements to "close" the SAT prover.

As far as I can tell this is sound (although very incomplete).

Implementation

We compare these strategies for the SAT prover along with two control strategies: `FirstAction` and `LeanCoPCut`.

1. `SATSolve` Plain SATCoP[4]. Last week's best model.
2. `SATSolveLiteral` SATCoP with the extension reordering.
3. `SATResetAtStaleAction` Reset when no new actions to the SAT solver.
4. `SATResetAtStaleBacktrack` Reset when backtracking if no new actions have been added to the SAT solver.
5. `SATResetBandit` Tiny contextual bandit choosing between `Backtrack(1)`, `Backtrack(2)`, and `Reset(n + 1)` depending on whether the shadow CNF is still growing.

Overall Results

The CRS3 tier is currently hammered, so most of these tests were done locally with a small 60 second time limit and 1 million step limit.

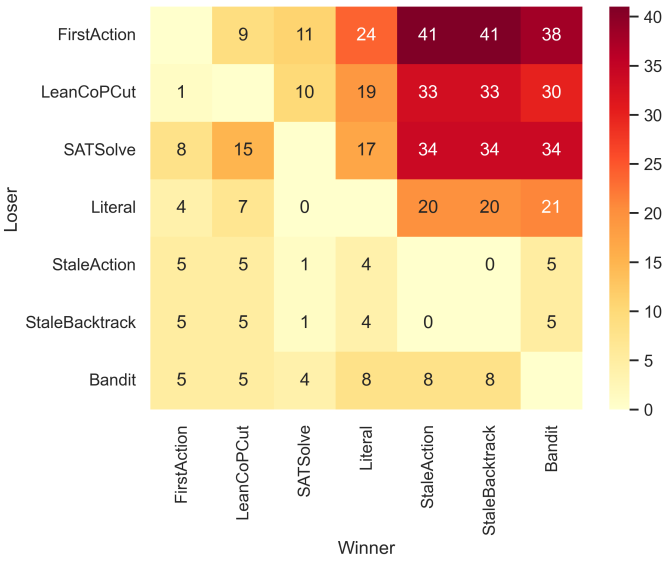


Figure 1: Winners and losers of the strategies in a representative subset of 501 problems by Chvalovský and Jakubův [1].

Some Interesting Results

References

[1] Karel Chvalovský and Jan Jakubův. "Characteristic Subsets of TPTP Benchmarks". In: *Proceedings of the 6th Conference on Artificial Intelligence and Theorem Proving (AITP 2021)* (Aussois, France, Sept. 5–10, 2021). Extended abstract in informal proceedings. 2021.

[2] Martin Fixman. "Using SAT Features to Guide Tableau Provers". (Last week's one-pager). Apr. 22, 2026.

[3] Jens Otten. "20 Years of leanCoP — an Overview of the Provers". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*. Vol. 3613. CEUR Workshop Proceedings. 2023.

[4] Michael Rawson and Giles Reger. "Eliminating Models During Model Elimination". In: *Automated Reasoning with Analytic Tableaux and Related Methods*. Ed. by Anupam Das and Sara Negri. Vol. 12842. Cham: Springer International Publishing, 2021, pp. 250–265. ISBN: 978-3-030-86058-5 978-3-030-86059-2.

Problem	FirstAction		LeanCoPCut		SATSolve		Literal		StaleAction		StaleBacktrack		Bandit	
	time	steps	time	steps	time	steps	time	steps	time	steps	time	steps	time	steps
CSR052+1.p	—		—		—		31s	35,616	1s	149	**1s**	**149**	1s	277
GEO185+3.p	—		—		—		—		**11s**	**3,899**	13s	3,900	17s	3,583
GEO202+3.p	—		26s	13,927	—		—		**7s**	**2,222**	7s	2,222	10s	2,399
GEO233+1.p	—		—		—		—		14s	3,219	14s	3,219	**7s**	**1,870**
GEO262+3.p	—		—		—		—		7s	1,802	**6s**	**1,802**	8s	2,696
HWV025-2.p	—		—		50s	7,392	35s	8,025	8s	1,151	**7s**	**1,151**	—	
KRS072+1.p	—		—		—		—		**3s**	**1,605**	4s	1,605	9s	3,028
KRS162+1.p	—		—		—		—		21s	4,486	22s	4,486	**19s**	**4,431**
KRS167+1.p	59s	30,732	—		15s	5,307	16s	15,277	5s	2,042	5s	2,042	**2s**	**658**
LCL126-1.p	—		—		—		24s	2,415	**3s**	**742**	3s	742	—	
NUM337+1.p	—		—		—		—		40s	5,185	**37s**	**5,185**	—	
NUM411+1.p	—		45s	26,538	—		—		6s	2,850	6s	2,850	**1s**	**659**
NUM432+3.p	11s	5,052	8s	4,010	—		60s	21,655	2s	914	2s	914	**2s**	**462**
RNG089+2.p	24s	5,771	23s	5,441	27s	4,476	11s	4,907	**1s**	**339**	2s	339	35s	4,025
SET027-7.p	—		—		—		—		49s	6,346	**42s**	**6,346**	55s	7,685
SET114-7.p	5s	1,632	5s	1,632	22s	4,698	11s	4,690	1s	123	**1s**	**123**	33s	3,739
SET602+4.p	—		—		—		—		—		—		**27s**	**3,503**
SET677+3.p	—		—		—		33s	11,136	1s	340	**1s**	**340**	2s	366
SEU121+2.p	—		43s	29,479	—		46s	42,182	11s	3,396	**7s**	**3,396**	19s	4,714
SEU234+1.p	—		—		31s	10,262	32s	30,860	14s	6,563	14s	6,563	**11s**	**4,236**
SWC136+1.p	—		—		—		—		**13s**	**2,982**	19s	2,982	16s	2,078
SWV908-1.p	—		—		—		—		**14s**	**1,286**	17s	1,286	22s	1,315
SYN154-1.p	—		—		38s	9,279	37s	18,150	1s	95	**1s**	**95**	2s	550
SYN158-1.p	—		—		—		33s	17,984	4s	1,394	4s	1,394	**3s**	**1,106**
SYN163-1.p	—		—		—		—		**6s**	**2,074**	6s	2,074	11s	2,827
SYN209-1.p	—		—		—		55s	26,905	**1s**	**481**	1s	481	4s	1,260
SYN260-1.p	7s	4,126	6s	4,126	32s	8,794	16s	9,156	**2s**	**845**	3s	845	10s	2,874
SYN625-1.p	—		—		—		42s	18,502	14s	2,312	**12s**	**2,312**	—	
SYN669-1.p	—		—		—		—		3s	1,082	3s	1,082	**2s**	**582**
SYN677-1.p	—		—		—		—		9s	2,841	8s	2,842	**2s**	**708**

Table 1: Thirty interesting problems for comparing these programs. All tested locally; step count should be accurate, time likely is not.